

Bibliothèques mathématiques: Numpy, Linalg,...

- Les tableaux (arrays) sont des objets fournis par le module numpy. Contrairement aux listes ils contiennent des objets de types identiques, comme des flottants, des complexes ou des entiers.
- L'instruction : `>>> from numpy import *` permet de charger en mémoire toute la bibliothèque numpy
- `>>> T = array([0.1, 3, -1, 4, 6.7])` permet de convertir la liste `[0.1, 3, -1, 4, 6.7]` en un tableau de type array.
- `>>> M=array([[0.4, 0.9, 4],[-1.5, 2, 1./3]])` permet de construire la matrice
`[0.4, 0.9, 4]`
`[-1.5, 2, 1./3]` à partir des deux listes `[0.4, 0.9, 4]` et `[-1.5, 2, 1./3]`.

La liste `[[0.4, 0.9, 4],[-1.5, 2, 1./3],[1,3,6]]` et la matrice `array([[ 0.4 , -1.5 , 1. ], [ 0.9 , 2. , 3. ], [ 4. , 0.33333333, 6. ]])` ont le même contenu cependant l'utilisation de la forme matricielle `numpy.array` permet d'exploiter plusieurs fonctions prédéfinies.

- `>>> import numpy`
- `>>> A=[[0.4, 0.9, 4],[-1.5, 2, 1./3],[1,3,6]]`
- `>>> B=numpy.array(A)`
- `>>> B.trace()`
8.400000000000000004
- `>>> B.transpose()`
array([[0.4 , -1.5 , 1.],
[0.9 , 2. , 3.],
[4. , 0.33333333, 6.]])
- `>>> A.trace()`
- Traceback (most recent call last):File "<console>", line 1, in
<module> AttributeError: 'list' object has no attribute 'trace'
- `>>> A.transpose()`

Traceback (most recent call last):File "<console>", line 1, in
<module> AttributeError: 'list' object has no attribute 'transpose'

- `>>> range(1,3,0.5)`

Traceback (most recent call last):File "<console>", line 1, in
<module> TypeError: 'float' object cannot be interpreted as an integer

- `>>> arange(1,3,0.5)`

`array([ 1. , 1.5, 2. , 2.5])`

- `linspace(a,b,n)` permet de créer un tableau dont les n composantes sont uniformément réparties entre a et b.
- `>>> from numpy import *`
- `>>> a=1;b=2;n=3;pas=(b-a)/(n-1)`
- `>>> linspace(a,b,n)`
`array([ 1. , 1.5, 2. ])`
- `>>> list(arange(a,b+pas,pas))`
`[1.0, 1.5, 2.0]`

- `>>> np.zeros(5)`
`array([ 0., 0., 0., 0., 0.])`
- `>>> np.ones((2,4))` # tableau 2d de taille 2 x 4 contenant des 1
`array([[1., 1., 1., 1.],
 [1., 1., 1., 1.]])`
- `>>> B=numpy.array([[0.4, 0.9, 4],[-1.5, 2, 1./3],[1,3,6]])`
- `>>> C=zeros_like(B)`
`array([[ 0., 0., 0.], [ 0., 0., 0.], [ 0., 0., 0.]])`
- `>>> D=ones_like(B)`
`array([[1., 1., 1.],
 [1., 1., 1.],
 [1., 1., 1.]])`

- `>>> from numpy import *`
- `>>> T=random.rand(4) # pour un tableau 1d de 4 éléments`
`T= array([ 0.05662015, 0.58904519, 0.72276614, 0.13775529])`
- `>>> T.dtype`
`dtype('float64')`
- `>>> X=random.randn(3,3) # pour une matrice aléatoire 3*3`
- `>>> X`
`array([[ 1.50331972, -0.56666019, -0.03043645],`
 `[ 0.86226869, 0.07233328, 0.28352836],`
 `[ 1.41434694, -0.67787242, 0.36466105]])`
- `>>> X=random.randint(0,9,5) # loi uniforme sur les entiers de`
`[0,9[ (donc [0,8])`
- `>>> #génération aléatoire de 5 entiers dans l'intervalle [0,8]`
- `>>> X`
`array([6, 7, 5, 5, 2])`

- `>>> X=random.randint(0,9,(3,3)) # pour obtenir une matrice 3*3 à valeurs entières dans [0,8]`

```
X= array([[3, 3, 5], [1, 2, 4], [2, 3, 1]])
```

- `>>> T=diag(X,k=0); print(T)`

```
[3  2  1]
```

- `>>> T=diag(X,k=1); print(T)`

```
[3  4]
```

- `>>> T=diag(X,k=-1); print(T)`

```
[1  3]
```

```
>>> M=eye(3) # matrice identité 3*3
```

```
M= array([[ 1.,  0.,  0.],  
          [ 0.,  1.,  0.],  
          [ 0.,  0.,  1.]])
```

- >>> T=numpy.array([3,2.6,8,7,9,100.5])
- >>> T.ndim
1
- >>> T.shape
(6,)
- >>> M=reshape(T,(3,2))
M=array([[3. , 2.6],
 [8. , 7.],
 [9. , 100.5]])
- >>> M[1][0]=44
- M=array([[3. , 2.6],
 [44. , 7.],
 [9. , 100.5]])
- >>> T
array([3. , 2.6, 44. , 7. , 9. , 100.5])

- `>>> M=random.randint(0,10,(4,4))`

```
array([[9, 5, 2, 9],  
       [4, 8, 0, 6],  
       [8, 8, 3, 6],  
       [9, 1, 7, 0]])
```

- `>>> M[:,1]`

```
array([5, 8, 8, 1])
```

- `>>> M[2,:]`

```
array([8, 8, 3, 6])
```

- `>>> M[2,3]`

6

- `>>> N=10*M`

```
>>> C=4*M+2*N
```

- `>>> C`

- `array([[216, 120, 48, 216],`

- `[ 96, 192, 0, 144],`

- `[192, 192, 72, 144],`

- `[216, 24, 168, 0]])`

- `>>> M=random.randint(0,10,(2,2))`
- `>>> N=random.randint(0,10,(2,2))`
- `>>> S=M+N`
- `>>> M`
`array([[4, 6],`
 `[6, 2]])`
- `>>> N`
`array([[4, 7],`
 `[2, 4]])`
- `>>> S`
`array([[ 8, 13],`
 `[ 8, 6]])`
- `>>> P=M*N` `# produit élément par élément`
- `>>> P`
`array([[16, 42],`
 `[12, 8]])`

- la multiplication matricielle s'obtient avec la fonction dot :
- `>>> A=random.randint(0,10,(2,2))`
- `>>> A`
- `array([[0, 6],
 [9, 9]])`
- `>>> B=random.randint(0,10,(2,2))`
- `>>> B`
- `array([[0, 5],
 [8, 2]])`
- `>>> C=dot(A,B)`
- `>>> C`
- `array([[48, 12],
 [72, 63]])`
- `>>> D=A.dot(B)`
- `>>> D`
- `array([[48, 12], [72, 63]])`

- produit extérieur : Si x est un tableau à 1 dimension de taille m et y un tableau à 1 dimension de taille n , $A = \text{outer}(x,y)$ renvoie un tableau à 2 dimensions de taille $m \times n$ avec $A_{i,j} = x_i y_j$.
- `>>> A=random.randint(1,10,6)`
- `>>> B=random.randint(1,10,4)`
- `>>> Z=outer(A,B)`
- `>>> A` `array([3, 3, 7, 4, 5, 9])`
- `>>> B` `array([4, 3, 2, 4])`
- `>>> Z`
- `array([[12, 9, 6, 12],`
- `[12, 9, 6, 12],`
- `[28, 21, 14, 28],`
- `[16, 12, 8, 16],`
- `[20, 15, 10, 20],`
- `[36, 27, 18, 36]])`

- Concaténation de tableaux, on utilise la fonction concatenate.
- `>>> A=random.randint(0,10,(2,2))`
- `>>> B=random.randint(0,10,(2,2))`
- `>>> C=concatenate((A,B),axis=0)`
- `>>> D=concatenate((A,B),axis=1)`
- `>>> A`
`array([[9, 0],`
`[2, 3]])`
- `>>> B`
`array([[2, 8],`
`[0, 0]])`
- `>>> C`
`array([[9, 0],`
`[2, 3],`
`[2, 8],`
`[0, 0]])`
- `>>> D`
`array([[9, 0, 2, 8],`
`[2, 3, 0, 0]])`
- `>>> D`
`array([[9, 0, 2, 8],`
`[2, 3, 0, 0]])`